Implement Client Script & UI Policy (Incident)

Project Documentation | ServiceNow Incident Management

1. Introduction

Project Title: Implement Client Script & UI Policy (Incident)

Team Member	Role
LAKSHANA J	Team Leader
BHARATHI K	Team Member
DHIVYADHARSHINI V	Team Member
ISHWARYA R	Team Member

2. Project Overview

Purpose: To use UI Policies and Client Scripts to enforce data integrity on Incident records in ServiceNow, so that agents enter complete, correct data before an incident is saved or submitted.

Key Features:

- Make fields mandatory based on conditions (e.g. Close notes when State = Resolved).
- Auto-populate field values (e.g. set Urgency/Impact defaults from Category).
- Control field behavior: show/hide, read-only and mandatory using UI Policy Actions.
- Form validation with Client Scripts (onLoad, onChange, onSubmit) and clear error messages.
- Block form submission when required conditions are not met.

3. Architecture

ServiceNow is a single platform, so the MERN-style layers map as follows:

Layer	Description
Frontend (Client)	Incident form in the Platform UI (browser). **UI Policies** and **UI Policy Actions** control mandatory / visible / read-only states without code. **Client Scripts** (JavaScript using g_form, g_user) handle onLoad, onChange, onSubmit and onCellEdit events.
Backend (Server)	ServiceNow instance (Glide server-side JavaScript). Handles saving records, ACLs, and optional Business Rules / Data Policies for server-side enforcement.
Database	Relational tables managed by the platform. Main table: **incident** (extends Task). Fields used: number, caller_id, category, subcategory, short_description, impact, urgency, priority, state, assignment_group, assigned_to, close_code, close_notes.

4. Setup Instructions

Prerequisites: ServiceNow Personal Developer Instance (PDI) from developer.servicenow.com; admin role; a modern browser (Chrome/Edge); basic JavaScript knowledge.

- Sign in to your PDI as admin and open *System Update Sets > Local Update Sets*; create and set a new update set for this project.
- Open *Incident > Create New* to view the form; use Form Layout to confirm required fields are present.
- Create UI Policies: *System UI > UI Policies > New* (Table: Incident, add Conditions and UI Policy Actions).
- Create Client Scripts: *System Definition > Client Scripts > New* (Table: Incident, choose Type, write script, tick Active).
- Save and test on the Incident form.

5. Folder Structure

ServiceNow stores configuration as records instead of files. Equivalent organization:

Area	Contents
Client	UI Policy (and UI Policy Actions) on **incident**; Client Scripts: onLoad, onChange (category, state, priority), onSubmit.
Server	Table **incident**, ACLs, Business Rules / Data Policies (future), Update Set holding all changes.

6. Running the Application

There is no npm start; the instance runs in the cloud. Open the PDI URL, log in, and go to *Incident > Create New* (or *Incident > All* to open an existing record). Changes to UI Policies and Client Scripts apply immediately after saving; reload the form to test.

7. API Documentation

Client-side (g_form) methods used:

Method	Purpose / Example
g_form.getValue('field')	Read a field value, e.g. g_form.getValue('state')
g_form.setValue('field', val)	Auto-populate, e.g. g_form.setValue('urgency','2')
g_form.setMandatory('field', true)	Make mandatory, e.g. close_notes

Method	Purpose / Example
g_form.setReadOnly / setVisible	Lock or show/hide a field
g_form.addErrorMessage(msg)	Show an error banner on validation failure
return false (onSubmit)	Stops the form from being submitted

```
function onSubmit() {
  if (g_form.getValue('state') == '6' && !g_form.getValue('close_notes')) {
    g_form.addErrorMessage('Close notes are required to resolve.');
    return false;
  }
}
```

REST Table API (for reference): `GET /api/now/table/incident?sysparm_limit=1` returns JSON such as `{"result":[{"number":"INC0010001","state":"1","short_description":"..."}]}`. Note that client-side policies do not run for REST calls.

8. Authentication

Users sign in with ServiceNow credentials (or SSO) and a server-side session is created. Authorization is role-based: ACLs and roles (e.g. itil, admin) decide who can read or write incident fields. REST clients use Basic Auth or OAuth tokens. Client Scripts can read the logged-in user via g_user (e.g. g_user.hasRole('itil')).

9. User Interface

Screenshot placeholder	Screenshot placeholder
Incident form with mandatory field indicators (UI Policy)	Error message shown when submit is blocked (Client Script)

10. Testing

Manual functional testing on the Incident form with different roles and conditions:

- Positive/negative cases: field becomes mandatory only when its condition is true; submit blocked when it is not met.
- Verify onChange auto-population, read-only and visibility behavior, and the error messages.
- Browser developer console (and Script Debugger) used to troubleshoot; ATF (Automated Test Framework) is optional.

11. Screenshots or Demo

[Insert screenshots of UI Policy list, UI Policy Actions, Client Script records, and a demo video link here.]

12. Known Issues

- UI Policies and Client Scripts run only in the browser; they are bypassed by imports, REST/SOAP and background scripts.
- Mandatory rules are not enforced server-side unless a Data Policy or Business Rule is added.
- Conflicting UI Policies and Client Scripts on the same field depend on execution order.

13. Future Enhancements

- Add Data Policies and Business Rules for server-side validation.
- Extend the logic to Service Catalog (Catalog Client Scripts / Catalog UI Policies) and Workspace forms.
- Automate regression tests with ATF and move work into a scoped application.